

# 计算的边界

高精度、位运算与离散化 · 7题 · C++ & Python 双语对照

小鲁在上个月的语法训练中，抱怨过"C++的int不够大"。小华说："今天你终于遇到了C++的短板——高精度计算。但先别急——这也正是Python最闪耀的时刻：int无精度上限，四则运算零负担。"本章的位运算部分教二进制思维——取最低位、计数、快速幂——底层优化与竞赛高频考点。离散化和区间合并则是空间压缩和贪心的经典案例。

## 本章角色

-  **小鲁**（进阶者）· 第一次深刻感受Python的震撼——高精度四题C++手写100行，Python四行
-  **小华**（算法队长）· 讲解lowbit(x&-x)的二进制原理和补码证明

### Python震撼教育：高精度与位运算

- 高精度加/减/乘/除**：C++手写30-50行 → Python: int(input()) (+ - \* //) 一行
- lowbit**：x & -x 取最低位的1。树状数组的核心操作
- bit\_count**：Python int.bit\_count() / C++ \_\_builtin\_popcount()
- 离散化**：收集坐标→排序去重→二分映射→前缀和。处理稀疏大范围的标准手段
- 区间合并**：按左端点排序→贪心遍历→合并重叠。O(n log n)

## NQ120 高精度加法 AcWing 791

"C++的int最大21亿，long long最大 $9 \times 10^{18}$ ——但如果你要算 $10^{200} + 10^{200}$ 呢？"小华严肃地说。"这就是高精度计算。C++需要手写大整数——用字符串读入，按位加，处理进位。"小鲁很惊讶："这么基础的操作还要手写？！"小华笑道："对。但这正是Python最震撼的地方——Python的int无精度上限，直接a+b就行。C++手写30行，Python一行。这就是'用对语言'的力量。"

**输入** 两行，每行一个正整数（长度≤100000位）。

**输出** 两数之和。

**范围** 数字长度 ≤ 100000

输入

123456789  
987654321

输出

1111111110

**思路** C++高精度加法：字符串读入→倒序存入vector→按位相加处理进位→倒序输出。核心代码：for i from 0 to max(len): t+=A[i]+B[i]; C.push\_back(t%10); t/=10。Python: print(int(input())+int(input()))——震撼教育时刻。

**技巧** Python的int底层用30-bit数组存储，自动扩展——可以处理任意大的整数（只受内存限制）。这是Python在算法竞赛中的最大优势之一——高精度计算零负担。C++手写高精度是理解数字表示和进位机制的好练习。

C++

```
#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6+7;

vector<int> add(vector<int> &A, vector<int> &B){
    vector<int> C;
    int t=0;
    for (int i=0; i<A.size()||i<B.size(); i++) {
        if (i<A.size()) t+=A[i];
        if (i<B.size()) t+=B[i];
        //对t取模，取进位
        C.push_back(t %10); //把t模10的余数入数组，商为进位
        t /= 10; //算出进位，供下一次循环使用
    }
    if(t) C.push_back(1); //最高位进位
    return C;
}

int main(){
    //输入为字符串
    string a,b;
    vector<int> A,B;

    cin >> a>>b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for(int i= a.size()-1; i>=0; i--) A.push_back(a[i]-
'0');
    for(int i= b.size()-1; i>=0; i--) B.push_back(b[i]-
'0');

    auto C = add(A,B);

    for(int i= C.size()-1; i>=0; i--) cout<<C[i]; //打印C
    return 0;
}
```

Python

```
# Python solution
```

## NQ121 高精度减法 AcWing 792

"加法有进位，减法有借位——高精度减法的核心是借位处理。"小华说，"首先判断A和B谁大，保证大减小。然后逐位减——如果不够减就向高位借1（当前位+10）。最后去掉前导0。C++手写减法比加法多一步：处理借位+前导0。Python——print(int(input())-int(input()))，又是零负担。"

**输入** 两行，每行一个正整数。

**输出** A-B的结果。

**范围** 数字长度  $\leq 100000$

输入

1000

1

输出

999

**思路** C++高精度减法：先判断 $A \geq B$ ，若A

**技巧** 借位处理：if A[i]

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;
bool isGreater(vector<int> &A, vector<int> &B)
{
    if (A.size() != B.size()) //比较大小
        return A.size() > B.size();
    //否则大小相等，逐位比较大小
    for (int i = A.size() - 1; i >= 0; i--)
        if (A[i] != B[i])
            return A[i] > B[i];
    return true; //每个数字都相同，长度相同，两数相等
}
//C= A-B
vector<int> sub(vector<int> &A, vector<int> &B)
{
    vector<int> C;
    //A是比较大的数
    int t = 0; //第一轮减法进位为0
    for (int i = 0; i < A.size(); i++)
    {
        t = A[i] - t;
        if (i < B.size())
            t -= B[i]; //A[i]-B[i] + t
        C.push_back((t + 10) % 10); //得到对应位
        if (t < 0)
            t = 1;
        else
            t = 0; //判断是否借一位
    }
    //去掉开头的0
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}

int main()
{
    //输入为字符串
    string a, b;
    vector<int> A, B;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; i--)
        B.push_back(b[i] - '0');
    //判断A与B的大小
    if (isGreater(A, B))
    {
        auto C = sub(A, B);
        for (int i = C.size() - 1; i >= 0; i--)
            cout << C[i]; //打印C
    }
    else
    {
        auto C = sub(B, A);
        cout << "-";
        for (int i = C.size() - 1; i >= 0; i--)
            cout << C[i]; //打印C
    }
}

```

Python

# Python solution

```

    }
    return 0;
}

```

## NQ122 二进制中1的个数 AcWing 801

"给你一个整数，求它的二进制表示中1的个数。"小华说，"这就是'汉明重量'——编译器优化和密码学中常用的操作。C++用\_\_builtin\_popcount()或while(x) cnt+=x&1, x>>=1。Python 3.8+用x.bit\_count()。还有lowbit技巧：x&-x取最低位的1，while(x) x-=x&-x, cnt++——比逐位检查更快。"

**输入** 一个整数n。

**输出** n的二进制表示中1的个数。

**范围**  $0 \leq n \leq 10^9$

| 输入 | 输出 |
|----|----|
| 5  | 2  |

**思路** 三种方法：1)逐位检查while(n): cnt+=n&1; n>>=1; 2)lowbit法while(n): n-=n&-n; cnt++——复杂度=1的个数而非位数；3)\_\_builtin\_popcount/Python bit\_count()一行搞定。lowbit=n&-n是位运算中最精妙的trick之一。

**技巧** x&-x取x的最低位的1——证明：负数用补码表示，-x=~x+1，x&~x+1就是最低位1的位置。这个trick在树状数组（Fenwick Tree）中是核心操作。bit\_count()是Python 3.8+新特性，C++20有std::popcount。

C++

```

#include <iostream>
using namespace std;
inline int lowbit(int &x)
{
    return x & -x; //x的二进制的最后一个1所构成的整数
}
int main()
{
    int n, x;
    cin >> n;
    while (n--)
    {
        //读入一个数，并且计算1的个数，并且输出
        cin >> x;
        int res = 0; //计数变量
        while (x)
            x -= lowbit(x), res++; //使用表达式，简化代码去
        括号
        cout << res << " "; //输出运算结果
    }
    return 0;
}

```

Python

```
# Python solution
```

"高精度乘高精度——C++手写要三重循环：A的每一位乘B的每一位，结果加到对应位置。"小华在黑板上演示。小鲁问："那乘法是不是比加减法难很多？""对——加减 $O(n)$ ，朴素乘法 $O(n^2)$ 。还有更快的Karatsuba和FFT算法可以在 $O(n \log n)$ 内完成，但那已经是大数运算的科研前沿了。Python——`print(int(input())*int(input()))`。"

**输入** 两行，每行一个正整数。

**输出** 乘积。

**范围** 数字长度  $\leq 100000$

| 输入 | 输出  |
|----|-----|
| 12 | 408 |
| 34 |     |

**思路** C++高精度乘法：两重循环  $C[i+j] += A[i] * B[j]$ ;  $C[i+j+1] += C[i+j] / 10$ ;  $C[i+j] \% = 10$ ; 最后去前导0。时间复杂度 $O(\text{len}(A) \times \text{len}(B))$ 。Python的int乘法底层用Karatsuba算法优化——对超长整数自动切换，比手写快得多。

**技巧** Python int乘法内部根据数字大小自动选择算法：小数字用基础乘法，大数字切到Karatsuba( $O(n^{1.585})$ )，超大数字用Toom–Cook。这就是为什么Python数据处理生态如此强大——底层的数学运算经过极致优化。

C++

```

#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
const int N = 1e6 + 7;

//C= A*b,t的用法很精彩!
vector<int> mul(vector<int> &A, int b){
    vector<int> C;
    int t=0;
    for(int i=0; i<A.size() || t; i++){
        //如果t还有剩余或者A还有剩余, 乘法结果C需要加新的位
        if(i<A.size()) t+=A[i]*b; //把b当作整体做乘法
        C.push_back(t % 10); //存储第i位
        t /=10; //更新t
    }
    while(C.size()>1 && C.back()==0) C.pop_back(); //去掉C
    开头的0
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b; //输入的b比较小, 作为一个整体处理
    vector<int> A,C;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');
    C=mul(A,b);
    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
    return 0;
}

```

Python

# Python solution

## NQ124 高精度除法 AcWing 794

"除法是四则运算中最复杂的——需要从高位到低位逐位试商。"小华说。高精度除以低精度相对简单（逐位除）；高精度除以高精度则需要二分法试商。"好消息是——Python `print(int(input())/int(input()))` 一行搞定，还能同时得到商和余数用 `divmod(a,b)`。高精度四则运算到此全部覆盖——C++手写超越100行，Python四行收工。"

输入

两行，每行一个正整数。

输出

第一行商，第二行余数。

范围

数字长度  $\leq 100000$ 

输入

10  
3

输出

3  
1

**思路** C++高精度除以低精度：从高位到低位， $r=r*10+A[i]$ ;  $C.push\_back(r/B)$ ;  $r\%=B$ 。高精度除以高精度需二分或减法模拟。Python:  $divmod(a,b)$ 返回(商,余数)元组。

**技巧** Python高精度总结：input(),int(),四则运算——零负担。C++手写高精度的意义在于理解底层运算原理——数字如何存储、进位借位如何处理。两种路径都有价值——但竞赛时用Python省下的时间可以做更多题。

C++

```
#include <iostream>
#include <cstring>
#include <vector>
#include <algorithm>

using namespace std;
const int N = 1e6 + 7;

//C= A/b 余数为r
vector<int> div(vector<int> &A, int b, int &r){
    vector<int> C;
    r=0;//余数
    for(int i=A.size()-1; i>=0; i--){
        r = r* 10 + A[i]; //从最高位开始，当前余数r需要上一次运算的r乘10
        C.push_back(r/b);
        r %= b;
    }
    reverse(C.begin(), C.end()); //定义的数是最低位在C[0],因此需要取逆
    while(C.size()>1 && C.back()==0) C.pop_back(); //去掉C开头的0字符
    return C;
}

int main()
{
    //输入为字符串
    string a;
    int b; //输入的b比较小，作为一个整体处理
    vector<int> A;

    cin >> a >> b; // a="786654"
    //倒序存储A, B, a[0]乃是最低位
    for (int i = a.size() - 1; i >= 0; i--)
        A.push_back(a[i] - '0');

    int r;
    auto C=div(A,b,r); //使用auto自动辨别变量类型

    for (int i = C.size() - 1; i >= 0; i--)
        cout << C[i]; //打印C
    cout<<endl<<r<<endl; //打印余数
    return 0;
}
```

Python

# Python solution

"有一个无限长的数轴，在几个坐标上加了值。现在要查询若干区间和。"小华说，"但坐标范围是 $-10^9$ 到 $10^9$ ——不能开 $10^9$ 大小的数组。怎么办？"离散化！把所有用到的坐标收集起来，排序去重，映射成0,1,2,...的连续下标。然后用前缀和——问题变成熟悉的区间查询。离散化就是'压缩稀疏坐标'的技术——只存储和处理用得到的数据。"

**输入** 第一行n,m。然后n行每行x c（位置加值）。然后m行每行l r（查询区间和）。

**输出** 每行查询结果。

**范围**  $-10^9 \leq x \leq 10^9$ ,  $n, m \leq 100000$

| 输入  | 输出 |
|-----|----|
| 3 3 | 8  |
| 1 2 | 0  |
| 3 6 | 5  |
| 7 5 |    |
| 1 3 |    |
| 4 6 |    |
| 7 8 |    |

**思路** 离散化三步骤：1)收集所有坐标(x, l, r)；2)排序去重→映射为0..k-1；3)在压缩后的数组上做前缀和+查询。用二分查找原坐标对应的压缩下标。Python用bisect\_left。时间 $O((n+2m)\log(n+2m))$ 。

**技巧** 离散化是处理稀疏大范围数据的标准手段——坐标0和 $10^9$ 之间的空间不需要全部存储。Python: `all_x=sorted(set(coords)); idx={v:i+1 for i,v in enumerate(all_x)}` (1-indexed)。结合前缀和，大范围区间查询轻松解决。

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int, int>的pair

const int N = 300007; //需要记录n个坐标值, 以及m个区间的坐标, 所以是100000*(1+2)
int n, m;
int a[N], s[N];

vector<int> alls; //存储所有的点坐标, 包括询问点坐标
vector<PII> add, query; //输入操作的坐标, 查询的区间坐标
//在alls之中, 查找值是x的元素值的下标, 并且映射到[1...n]
int find(int x)
{
    int left = 0, right = alls.size() - 1; //vector从0开始计数
    while (left < right)
    {
        int mid = left + right >> 1; //取中值
        if (alls[mid] >= x)
            right = mid;
        else
            left = mid + 1;
    }
    return right + 1; //把坐标值映射到1, 2, 3...而不是从0开始
}

///? 使用离散化的方式来预处理输入数据
///? 把输入的数字映射成{下标, 数字}的pair, 然后再用差分的方法计算区间和
///? 本题目可以参考797.差分的模板写成
int main()
{
    cin >> n >> m;
    ///!离散化: 构造映射表

    for (int i = 0; i < n; i++)
    {
        ///todo 不直接把数存入a[i], 乃是存入add容器内
        int x, c;
        cin >> x >> c;
        add.push_back({x, c}); ///?将某一位置x上的数加c
        alls.push_back(x); //把x位置加入alls
    }
    // 读入query
    for (int i = 0; i < m; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l); //坐标与alls的下标做映射
        alls.push_back(r);
    }
    //alls是下标映射表, 需要去重
    //使用STL算法库的sort和unique
    sort(alls.begin(), alls.end());
    //unique返回不重复区间的下一个元素位置, 用erase去掉尾部
    alls.erase(unique(alls.begin(), alls.end()), alls.end());
}

```

Python

# Python solution

```

//针对离散化的数据，计算区间和
for (auto it : add) //用c++11语法遍历add vector
{
    //初始化a[i],用离散化后的坐标
    int index = find(it.first);
    a[index] += it.second; //将index位置上的数加c
}
//预处理前缀和
for (int i = 1; i <= alls.size(); i++)
    s[i] = s[i - 1] + a[i];
//处理询问，并且输出
for (auto item : query)
{
    int l = find(item.first), r = find(item.second);
    cout << s[r] - s[l - 1] << endl; //前缀和计算公式
}
return 0;
}

```

## NQ126 区间合并 AcWing 803

"最后一题——把重叠的区间合并。"比如[1,3]和[2,6]重叠→合并成[1,6]。"小华说，"贪心法：按左端点排序，然后遍历。维护当前合并区间[st,ed]——如果新区间的左端点 $\leq$ ed说明重叠，更新 $ed = \max(ed, new\_ed)$ ；否则当前区间完成，开启新区间。一次排序 $O(n \log n)$ +一次遍历 $O(n)$ ——总共 $O(n \log n)$ 。"

**输入** 第一行n。然后n行每行l r表示区间。

**输出** 第一行合并后区间数。然后每行一个区间。

**范围**  $-10^9 \leq l \leq r \leq 10^9$ ,  $n \leq 100000$

| 输入  | 输出  |
|-----|-----|
| 5   | 3   |
| 1 2 | 1 4 |
| 2 4 | 5 6 |
| 5 6 | 7 9 |
| 7 8 |     |
| 7 9 |     |

**思路** 区间合并：1)按左端点升序排序；2)初始化st,ed为第一个区间；3)遍历剩余：如果 $l \leq ed$ 则 $ed = \max(ed, r)$ （合并）；否则输出[st,ed]并开始新区间。注意输入区间可能不完全排序——必须先排序。时间 $O(n \log n)$ 。

**技巧** 贪心的核心：按左端点排序后，重叠区间一定连续出现。边界条件ed

C++

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
typedef pair<int, int> PII; //自定义类型PII, 表示容器类型是<int,int>的pair

const int N = 100007; //1<=n<=100000
int n; //n个区间

vector<PII> segments; //区间坐标

void merge(vector<PII> &segs)
{
    vector<PII> res;
    //pair排序先按左端点, 后右端点
    sort(segs.begin(), segs.end());

    //数字范围是-10^9<num<10^9, 因此负无穷大初始化为-2e9
    int left = -2e9, right = -2e9; //当前操作的区间[left, right]

    for (auto seg : segs) //遍历segs
        if (right < seg.first) //区间没有交集, 更新当前区间
        {
            //存储合并完毕的区间[left, right]
            if (left != -2e9) res.push_back({left, right});

            //更新当前操作区间
            left = seg.first, right = seg.second;
        }
        else right = max(right, seg.second); //更新右端点, 扩展区间

    if (left != -2e9) res.push_back({left, right}); //把最后一个区间入栈

    segs = res; //更新segs
}

int main()
{
    cin >> n;
    // 读入n组区间
    for (int i = 0; i < n; i++)
    {
        int l, r; //区间l和r
        cin >> l >> r;
        segments.push_back({l, r});
    }
    //合并区间segment
    merge(segments);
    //输出合并后的区间个数
    cout<<segments.size()<<endl; //输出合并后的区间元素个数

    return 0;
}

```

Python

# Python solution

## 本章知识点总结

- **高精度四则运算**：C++手写理解底层（进位/借位/试商），Python直接用——选对语言事半功倍
- **位运算核心**：lowbit= $x \& -x$ , bit\_count, 左移右移——底层优化的利器
- **离散化**：稀疏空间→压缩→常规操作——大数据+稀疏坐标的标准处理流程
- **区间合并**：排序+贪心遍历—— $O(n \log n)$ 解决最小区间覆盖问题

— 第11章完 · 共7题 · 自强不息 —